오류 처리 가이드

개요

AMD SEV 라이브러리는 다양한 계층에서 발생할 수 있는 오류들을 체계적으로 분류하고 처리할 수 있는 포괄적인 오류 처리 시스템을 제공합니다. 이 가이드는 일반적인 오류 상황, 원인, 해결 방법, 그리고 효과적인 디버깅 전략을 설명합니다.

오류 계층 구조

1. 최상위 오류 타입: `UserApiError`

```
pub  enum UserApiError {

FirmwareError(FirmwareError), // 펌웨어 관련 오류

IOError(io::Error), // I/O 관련 오류

ApiError(CertError), // 인증서 관련 오류

VmmError(VmmError), // VMM 관련 오류

UuidError(uuid::Error), // UUID 파싱 오류

HashstickError(HashstickError), // VLEK Hashstick 오류

VmplError, // VMPL 관련 오류

AttestationReportError(AttestationReportError), // 증명 보고서 오류

Unknown, // 알 수 없는 오류

}
```

2. 펌웨어 오류: `FirmwareError`

```
pub enum FirmwareError {
KnownSevError(SevError), // 알려진 SEV 오류
UnknownSevError(u32), // 알 수 없는 SEV 오류
IoError(std::io::Error), // I/O 오류
}
```

3. SEV 플랫폼 오류: `SevError`

```
pub enum SevError {
InvalidPlatformState = 0x0001, // 플랫폼 상태 오류
InvalidGuestState = 0x0002, // 게스트 상태 오류
InvalidConfig = 0x0003, // 설정 오류
InvalidLen = 0x0004, // 길이 오류
AlreadyOwned = 0x0005, // 이미 소유됨
InvalidCertificate = 0x0006, // 인증서 오류
PolicyFailure = 0x0007, // 정책 실패
Inactive = 0x0008, // 비활성 상태
InvalidAddress = 0x0009, // 주소 오류
BadSignature = 0x000A, // 서명 오류
BadMeasurement = 0x000B, // 측정 오류
// ... 더 많은 오류 코드들
```

```
}
```

일반적인 오류 상황 및 해결 방법

1. 플랫폼 관리 오류

1.1 플랫폼 상태 오류 (`InvalidPlatformState`)

발생 상황:

- 플랫폼이 초기화되지 않은 상태에서 작업 수행
- 잘못된 순서로 API 호출

해결 방법:

```
use sev::{
firmware::host::{Firmware, State},
error::{UserApiError, FirmwareError, SevError},
};

fn handle_platform_state_error() -> Result<(), UserApiError> {
let mut firmware = Firmware::open()
.map_err(|e| UserApiError::IOError(e))?;
// 플랫폼 상태 확인
let status = firmware.platform_status()?;
match status.state {
State::Uninitialized => {
```

```
println!("플랫폼이 초기화되지 않음. 초기화를 진행합니다...");

// 플랫폼 초기화 시퀀스

firmware.platform_reset()?;

firmware.pek_generate()?;

firmware.pdh_generate()?;

println!("플랫폼 초기화 완료");

},

State::Initialized => {

println!("플랫폼이 초기화됨. 작업을 계속할 수 있습니다.");

},

State::Working => {

println!("플랫폼이 작업 중. {} 게스트가 활성화됨", status.guests);

},

}

Ok(())

}



// 오류 처리 예제

fn  safe_platform_operation() {

match  handle_platform_state_error() {

Ok(()) => println!("플랫폼 작업 성공"),

Err(UserApiError::FirmwareError(FirmwareError::KnownSevError(SevError::InvalidPl

eprintln!("플랫폼 상태 오류: 플랫폼을 먼저 초기화하세요");
```

```
// 복구 로직 실행
},
Err(e) => eprintln!("기타 오류: {}", e),
}
}
```

1.2 권한 오류 (`IOError`)

발생 상황:

- `/dev/sev` 또는 `/dev/kvm` 디바이스에 대한 접근 권한 부족

해결 방법:

```
use std::fs::OpenOptions;
use sev::error::UserApiError;

fn check_device_permissions() -> Result<(), UserApiError> {
// SEV 디바이스 접근 테스트
match OpenOptions::new().read(true).write(true).open("/dev/sev") {
Ok(_) => println!("SEV 디바이스 접근 가능"),
Err(e) => {
match e.kind() {
std::io::ErrorKind::PermissionDenied => {
eprintln!("권한 오류: /dev/sev에 대한 읽기/쓰기 권한이 필요합니다");
eprintln!("해결 방법:");
```

```
eprintln!(" sudo chmod 666 /dev/sev");
eprintln!(" 또는 사용자를 kvm 그룹에 추가:");
eprintln!(" sudo usermod -a -G kvm $USER");
return Err(UserApiError::IOError(e));
},
std::io::ErrorKind::NotFound => {
eprintln!("디바이스 없음: /dev/sev 디바이스를 찾을 수 없습니다");
eprintln!("SEV가 지원되는 시스템인지 확인하세요");
return Err(UserApiError::IOError(e));
},
_ => {
eprintln!("I/O 오류: {}", e);
return Err(UserApiError::IOError(e));
}
}
}
}
// KVM 디바이스 접근 테스트
match OpenOptions::new().read(true).write(true).open("/dev/kvm") {
Ok(_) => println!("KVM 디바이스 접근 가능"),
Err(e) => {
eprintln!("KVM 디바이스 오류: {}", e);
eprintln!("KVM 모듈이 로드되어 있는지 확인하세요: lsmod | grep kvm");
```

```
return Err(UserApiError::IOError(e));
}
}
Ok(())
}
```

1.3 플랫폼 소유권 오류 (`AlreadyOwned`)

발생 상황:

- 플랫폼이 이미 다른 소유자에 의해 소유됨

해결 방법:

```
use sev::{
firmware::host::Firmware,
error::{UserApiError, FirmwareError, SevError},
};

fn handle_ownership_error() -> Result<(), UserApiError> {
let mut firmware = Firmware::open()
.map_err(|e| UserApiError::IOError(e))?;
// PEK 생성 시도
match firmware.pek_generate() {
Ok(()) => {
println!("PEK 생성 성공");
},
```

```
Err(UserApiError::FirmwareError(FirmwareError::KnownSevError(SevError::AlreadyOw

println!("플랫폼이 이미 소유되어 있습니다.");

println!("해결 방법:");

println!("1. 플랫폼 리셋 (모든 게스트 종료됨):");

println!(" firmware.platform_reset()?");

println!("2. 기존 소유자 인증서 사용");

// 사용자 선택에 따른 처리

println!("플랫폼을 리셋하시겠습니까? (y/N)");

let  mut  input = String::new();

std::io::stdin().read_line(&mut  input).unwrap();

if  input.trim().to_lowercase() == "y" {

firmware.platform_reset()?;

firmware.pek_generate()?;

println!("플랫폼 리셋 및 PEK 생성 완료");

} else {

println!("기존 PEK를 사용합니다.");

}

},

Err(e) => return Err(e),

}

Ok(())

}
```

2. 게스트 런치 오류

2.1 게스트 상태 오류 (`InvalidGuestState`)

발생 상황:

- 게스트가 잘못된 상태에서 작업 수행
- 런치 시퀀스가 올바르지 않음

해결 방법:

```
use sev::{
launch::sev::{Launcher, New},
error::{UserApiError, FirmwareError, SevError},
};
use std::fs::OpenOptions;

fn safe_guest_launch() -> Result<(), Box<dyn std::error::Error>> {
let kvm_fd = OpenOptions::new()
.read(true)
.write(true)
.open("/dev/kvm")?;
let sev_fd = OpenOptions::new()
.read(true)
.write(true)
.open("/dev/sev")?;
// 런처 생성 시도
```

```
let launcher: Launcher<New, _, _> = match Launcher::new(kvm_fd, sev_fd) {
Ok(launcher) => {
println!("런처 생성 성공");
launcher
},
Err(FirmwareError::KnownSevError(SevError::InvalidGuestState)) => {
eprintln!("게스트 상태 오류: 이전 게스트가 완전히 정리되지 않았습니다");
eprintln!("해결 방법:");
eprintln!("1. 잠시 대기 후 재시도");
eprintln!("2. 플랫폼 리셋");
// 재시도 로직
std::thread::sleep(std::time::Duration::from_secs(1));
let kvm_fd = OpenOptions::new()
.read(true)
.write(true)
.open("/dev/kvm")?;
let sev_fd = OpenOptions::new()
.read(true)
.write(true)
.open("/dev/sev")?;
Launcher::new(kvm_fd, sev_fd)?
},
Err(e) => return Err(e.into()),
```

```
};
println!("게스트 런치 준비 완료");
Ok(())
}
```

2.2 정책 실패 (`PolicyFailure`)

발생 상황:

- 게스트 정책이 플랫폼 정책과 호환되지 않음
- 최소 펌웨어 버전 요구사항 불충족

해결 방법:

```
use sev::{
launch::sev::{Policy, PolicyFlags},
firmware::host::{Firmware, Version},
error::{UserApiError, FirmwareError, SevError},
};

fn create_compatible_policy() -> Result<Policy, UserApiError> {
let mut firmware = Firmware::open()
.map_err(|e| UserApiError::IOError(e))?;
// 플랫폼 상태 확인
let status = firmware.platform_status()?;
let platform_version = status.build.version;
println!("플랫폼 펌웨어 버전: {}.{}",
```

```
platform_version.major, platform_version.minor);

// 호환 가능한 정책 생성

let  policy = Policy {

flags: PolicyFlags::NO_DEBUG, // 보수적인 정책

minfw: Version {

major: std::cmp::min(platform_version.major, 1),

minor: 0,

},

};

println!("생성된 정책:");

println!(" - 디버그 비활성화: {}", policy.flags.contains(PolicyFlags::NO_DEBUG));

println!(" - 최소 펌웨어 버전: {}.{}", policy.minfw.major, policy.minfw.minor);

Ok(policy)

}


fn  handle_policy_failure() {

match  create_compatible_policy() {

Ok(policy) => {

println!("호환 가능한 정책 생성 완료");

// 정책을 사용하여 게스트 런치 진행

},

Err(UserApiError::FirmwareError(FirmwareError::KnownSevError(SevError::PolicyFai

eprintln!("정책 실패: 요청된 정책이 플랫폼과 호환되지 않습니다");
```

```
eprintln!("해결 방법:");
eprintln!("1. 최소 펌웨어 버전 요구사항 낮추기");
eprintln!("2. 정책 플래그 조정");
eprintln!("3. 플랫폼 펌웨어 업데이트");
},
Err(e) => eprintln!("기타 오류: {}", e),
}
}
```

3. 인증서 관련 오류

3.1 인증서 검증 실패 (`InvalidCertificate`)

발생 상황:

- 손상된 인증서
- 만료된 인증서
- 잘못된 인증서 체인

해결 방법:

```
use sev::{
firmware::host::Firmware,
certs::sev::{sev::Chain, Verifiable},
error::{UserApiError, FirmwareError, SevError},
};
```

```
fn verify_and_fix_certificates() -> Result<Chain, UserApiError> {
let mut firmware = Firmware::open()
.map_err(|e| UserApiError::IOError(e))?;
// 인증서 체인 내보내기
let chain = match firmware.pdh_cert_export() {
Ok(chain) => chain,
Err(UserApiError::FirmwareError(FirmwareError::KnownSevError(SevError::InvalidCe
eprintln!("인증서 내보내기 실패: 인증서가 손상되었습니다");
eprintln!("해결 방법: 새로운 인증서 생성");
// 새 인증서 생성
firmware.pek_generate()?;
firmware.pdh_generate()?;
firmware.pdh_cert_export()?
},
Err(e) => return Err(e),
};
// 인증서 체인 검증
match chain.verify() {
Ok(()) => {
println!("✅ 인증서 체인 검증 성공");
},
Err(e) => {
```

```
eprintln!("❌ 인증서 체인 검증 실패: {}", e);
eprintln!("가능한 원인:");
eprintln!("1. 인증서 체인이 불완전함");
eprintln!("2. 서명이 유효하지 않음");
eprintln!("3. 인증서가 만료됨");
// 복구 시도
eprintln!("새로운 인증서 체인 생성을 시도합니다...");
firmware.platform_reset()?;
firmware.pek_generate()?;
firmware.pdh_generate()?;
let  new_chain = firmware.pdh_cert_export()?;
new_chain.verify().map_err(|e| {
eprintln!("새 인증서 체인도 검증 실패: {}", e);
UserApiError::ApiError(sev::error::CertError::UnknownError)
})?;
println!("✅ 새 인증서 체인 생성 및 검증 성공");
return Ok(new_chain);
}
}
Ok(chain)
}
```

4. SEV-SNP 특화 오류

4.1 RMP 관련 오류

발생 상황:

- RMP (Reverse Map Table) 초기화 실패
- 페이지 상태 불일치

해결 방법:

```
use sev::{
firmware::host::Firmware,
error::{UserApiError, FirmwareError, SevError},
};

fn handle_rmp_errors() -> Result<(), UserApiError> {
let mut firmware = Firmware::open()
.map_err(|e| UserApiError::IOError(e))?;
match firmware.snp_platform_status() {
Ok(status) => {
println!("SNP 플랫폼 상태: {}", status.state);
},
Err(UserApiError::FirmwareError(FirmwareError::KnownSevError(SevError::RMPInitRe
eprintln!("RMP 초기화 필요");
eprintln!("해결 방법:");
eprintln!("1. 시스템 재부팅");
eprintln!("2. SNP 모듈 재로드");
```

```
eprintln!(" sudo modprobe -r kvm_amd");

eprintln!(" sudo modprobe kvm_amd");

return Err(UserApiError::FirmwareError(

FirmwareError::KnownSevError(SevError::RMPInitRequired)

));

},

Err(UserApiError::FirmwareError(FirmwareError::KnownSevError(SevError::RMPInitFa

eprintln!("RMP 초기화 실패");

eprintln!("가능한 원인:");

eprintln!("1. 메모리 부족");

eprintln!("2. 하드웨어 문제");

eprintln!("3. 펌웨어 버그");

return Err(UserApiError::FirmwareError(

FirmwareError::KnownSevError(SevError::RMPInitFailed)

));

},

Err(e) => return Err(e),

}

Ok(())

}
```

4.2 TCB 버전 오류

발생 상황:

- TCB 버전이 요구사항보다 낮음
- 펌웨어 롤백 방지

해결 방법:

```
use sev::{
firmware::host::{Firmware, Config, TcbVersion, MaskId},
error::{UserApiError, FirmwareError, SevError},
};


fn handle_tcb_version_errors() -> Result<(), UserApiError> {
let mut firmware = Firmware::open()
.map_err(|e| UserApiError::IOError(e))?;
let status = firmware.snp_platform_status()?;
println!("현재 TCB 버전:");
println!(" 플랫폼: {:?}", status.platform_tcb_version);
println!(" 보고됨: {:?}", status.reported_tcb_version);
// 안전한 설정 생성
let safe_config = Config::new(
TcbVersion::new(
Some(status.platform_tcb_version.microcode),
status.platform_tcb_version.snp,
status.platform_tcb_version.tee,
status.platform_tcb_version.bootloader,
```

```
0,
),
MaskId(0),
);
match  firmware.snp_set_config(safe_config) {
Ok(()) => {
println!("SNP 설정 성공");
// 설정 커밋
match  firmware.snp_commit() {
Ok(()) => println!("SNP 설정 커밋 성공"),
Err(UserApiError::FirmwareError(FirmwareError::KnownSevError(SevError::BadVersio
eprintln!("펌웨어 버전 롤백 방지 오류");
eprintln!("현재 펌웨어 버전이 커밋된 버전보다 낮습니다");
eprintln!("해결 방법: 펌웨어 업데이트");
},
Err(e) => return Err(e),
}
},
Err(UserApiError::FirmwareError(FirmwareError::KnownSevError(SevError::BadSvn)))
eprintln!("SVN (Security Version Number) 오류");
eprintln!("제공된 이미지의 SVN이 커밋된 SVN보다 낮습니다");
eprintln!("해결 방법:");
eprintln!("1. 더 높은 SVN을 가진 이미지 사용");
```

```
eprintln!("2. 플랫폼 리셋 (보안 위험 있음)");
},
Err(e) => return Err(e),
}
Ok(())
}
```

종합적인 오류 처리 패턴

1. 계층적 오류 처리

```
use sev::error::{UserApiError, FirmwareError, SevError};
use std::error::Error;

fn  comprehensive_error_handler(error: UserApiError) {
println!("=== 오류 분석 ===");
match &error {
UserApiError::FirmwareError(fw_err) => {
println!("펌웨어 오류 발생");
match  fw_err {
FirmwareError::KnownSevError(sev_err) => {
println!("알려진 SEV 오류: {}", sev_err);
handle_sev_error(*sev_err);
```

```
},
FirmwareError::UnknownSevError(code) => {
println!("알 수 없는 SEV 오류 코드: 0x{:x}", code);
println!("AMD SEV 문서를 참조하거나 지원팀에 문의하세요");
},
FirmwareError::IoError(io_err) => {
println!("I/O 오류: {}", io_err);
handle_io_error(io_err);
},
}
},
UserApiError::IOError(io_err) => {
println!("I/O 오류: {}", io_err);
handle_io_error(io_err);
},
UserApiError::ApiError(cert_err) => {
println!("인증서 API 오류: {}", cert_err);
handle_cert_error(cert_err);
},
UserApiError::VmmError(vmm_err) => {
println!("VMM 오류: {}", vmm_err);
handle_vmm_error(vmm_err);
},
```

```
UserApiError::HashstickError(hs_err) => {
println!("VLEK Hashstick 오류: {}", hs_err);
handle_hashstick_error(hs_err);
},
UserApiError::VmplError => {
println!("VMPL 오류: 잘못된 VM Permission Level");
},
UserApiError::AttestationReportError(ar_err) => {
println!("증명 보고서 오류: {}", ar_err);
},
UserApiError::UuidError(uuid_err) => {
println!("UUID 파싱 오류: {}", uuid_err);
},
UserApiError::Unknown => {
println!("알 수 없는 오류");
},
}
// 오류 체인 출력
println!("\n=== 오류 체인 ===");
let mut source = error.source();
let mut level = 1;
while let Some(err) = source {
println!("레벨 {}: {}", level, err);
```

```
source = err.source();
level += 1;
}
}

fn handle_sev_error(sev_error: SevError) {
match sev_error {
SevError::InvalidPlatformState => {
println!("해결책: 플랫폼 상태를 확인하고 필요시 초기화하세요");
},
SevError::InvalidGuestState => {
println!("해결책: 게스트 상태를 확인하고 올바른 순서로 API를 호출하세요");
},
SevError::AlreadyOwned => {
println!("해결책: 플랫폼 리셋 또는 기존 소유자 인증서 사용");
},
SevError::PolicyFailure => {
println!("해결책: 정책 설정을 플랫폼 능력에 맞게 조정하세요");
},
SevError::InvalidLen => {
println!("해결책: 버퍼 크기를 확인하고 적절한 크기로 조정하세요");
},
_ => {
```

```
println!("일반적인 해결책:");
println!("1. 작업을 재시도하세요");
println!("2. 플랫폼 상태를 확인하세요");
println!("3. 시스템 로그를 확인하세요");
}
}
}

fn handle_io_error(io_error: &std::io::Error) {
match io_error.kind() {
std::io::ErrorKind::PermissionDenied => {
println!("해결책: 디바이스 권한을 확인하세요");
println!(" sudo chmod 666 /dev/sev /dev/kvm");
},
std::io::ErrorKind::NotFound => {
println!("해결책: 디바이스 파일이 존재하는지 확인하세요");
println!(" ls -la /dev/sev /dev/kvm");
},
std::io::ErrorKind::Interrupted => {
println!("해결책: 작업을 재시도하세요");
},
_ => {
println!("해결책: 시스템 상태를 확인하고 재시도하세요");
```

```
}
}
}

fn handle_cert_error(cert_error: &sev::error::CertError) {
match cert_error {
sev::error::CertError::InvalidGUID => {
println!("해결책: GUID 형식을 확인하세요");
},
sev::error::CertError::PageMisalignment => {
println!("해결책: 인증서 버퍼를 4K 페이지에 정렬하세요");
},
sev::error::CertError::BufferOverflow => {
println!("해결책: 버퍼 크기를 늘리세요");
},
sev::error::CertError::EmptyCertBuffer => {
println!("해결책: 호스트에서 인증서를 제공했는지 확인하세요");
},
_ => {
println!("해결책: 인증서 데이터를 다시 확인하세요");
}
}
}
```

```
fn handle_vmm_error(vmm_error: &sev::error::VmmError) {
match vmm_error {
sev::error::VmmError::InvalidCertificatePageLength => {
println!("해결책: 인증서 페이지 수를 늘리세요");
},
sev::error::VmmError::RateLimitRetryRequest => {
println!("해결책: 잠시 대기 후 재시도하세요");
println!(" std::thread::sleep(Duration::from_millis(100));");
},
_ => {
println!("해결책: VMM 설정을 확인하세요");
}
}
}

fn handle_hashstick_error(hs_error: &sev::error::HashstickError) {
match hs_error {
sev::error::HashstickError::InvalidLength => {
println!("해결책: Hashstick이 정확히 432바이트인지 확인하세요");
},
sev::error::HashstickError::EmptyHashstickBuffer => {
println!("해결책: 유효한 Hashstick 데이터를 제공하세요");
```

```
},
sev::error::HashstickError::InvalidReservedField => {
println!("해결책: Hashstick의 예약 필드가 올바른지 확인하세요");
},
_ => {
println!("해결책: AMD에서 제공받은 유효한 VLEK Hashstick을 사용하세요");
}
}
}
```

2. 재시도 로직

```
use std::time::{Duration, Instant};
use sev::error::{UserApiError, FirmwareError, SevError};

struct RetryConfig {
max_attempts: u32,
base_delay: Duration,
max_delay: Duration,
backoff_multiplier: f64,
}

impl Default for RetryConfig {
```

```rust
fn default() -> Self {
Self {
max_attempts: 3,
base_delay: Duration::from_millis(100),
max_delay: Duration::from_secs(5),
backoff_multiplier: 2.0,
}
}
}

fn retry_with_backoff<F, T>(
operation: F,
config: RetryConfig,
operation_name: &str,
) -> Result<T, UserApiError>
where
F: Fn() -> Result<T, UserApiError>,
{
let mut attempt = 1;
let mut delay = config.base_delay;
loop {
println!("시도 {}/{}: {}", attempt, config.max_attempts, operation_name);
match operation() {
```

```
Ok(result) => {
if  attempt > 1 {
println!("✅ {}번째 시도에서 성공", attempt);
}
return Ok(result);
},
Err(error) => {
if  attempt >= config.max_attempts {
eprintln!("❌ 모든 재시도 실패: {}", error);
return Err(error);
}
// 재시도 가능한 오류인지 확인
if !is_retryable_error(&error) {
eprintln!("❌ 재시도 불가능한 오류: {}", error);
return Err(error);
}
eprintln!("⚠️ 시도 {} 실패: {}. {}ms 후 재시도...",
attempt, error, delay.as_millis());
std::thread::sleep(delay);
// 지수 백오프
delay = std::cmp::min(
Duration::from_millis(
(delay.as_millis() as f64 * config.backoff_multiplier) as u64
```

```
),
config.max_delay,
);
attempt += 1;
}
}
}
}

fn is_retryable_error(error: &UserApiError) -> bool {
match error {
UserApiError::FirmwareError(FirmwareError::KnownSevError(sev_err)) => {
match sev_err {
SevError::HardwarePlatform => true, // 하드웨어 일시적 문제
SevError::RbModeExited => true, // Ring Buffer 모드 종료
SevError::ResourceLimit => true, // 리소스 부족 (일시적)
SevError::InvalidGuestState => true, // 게스트 상태 (일시적)
_ => false,
}
},
UserApiError::VmmError(sev::error::VmmError::RateLimitRetryRequest) => true,
UserApiError::IOError(io_err) => {
match io_err.kind() {
```

```
std::io::ErrorKind::Interrupted => true,
std::io::ErrorKind::WouldBlock => true,
std::io::ErrorKind::TimedOut => true,
_ => false,
}
},
_ => false,
}
}

// 사용 예제
fn robust_platform_operation() -> Result<(), UserApiError> {
retry_with_backoff(
|| {
let mut firmware = sev::firmware::host::Firmware::open()
.map_err(|e| UserApiError::IOError(e))?;
firmware.platform_status()?;
Ok(())
},
RetryConfig::default(),
"플랫폼 상태 조회",
)
```

```
}
```

3. 로깅 및 디버깅

```
use log::{debug, info, warn, error};

fn setup_logging() {
env_logger::Builder::from_default_env()
.filter_level(log::LevelFilter::Debug)
.init();
}

fn debug_sev_operation<F, T>(operation: F, operation_name: &str) -> Result<T, U
where
F: FnOnce() -> Result<T, UserApiError>,
{
info!("시작: {}", operation_name);
let start_time = std::time::Instant::now();
match operation() {
Ok(result) => {
let duration = start_time.elapsed();
info!("성공: {} (소요시간: {:?})", operation_name, duration);
Ok(result)
```

```
},
Err(error) => {
let  duration = start_time.elapsed();
error!("실패: {} (소요시간: {:?})", operation_name, duration);
error!("오류 상세: {}", error);
// 디버그 정보 수집
debug!("시스템 정보:");
debug!(" /dev/sev 존재: {}", std::path::Path::new("/dev/sev").exists());
debug!(" /dev/kvm 존재: {}", std::path::Path::new("/dev/kvm").exists());
// 커널 모듈 상태 확인
if  let Ok(output) = std::process::Command::new("lsmod")
.arg("kvm_amd")
.output()
{
debug!("kvm_amd 모듈: {}",
String::from_utf8_lossy(&output.stdout));
}
Err(error)
}
}
}


// 사용 예제
```

```rust
fn main() -> Result<(), Box<dyn std::error::Error>> {
setup_logging();
let result = debug_sev_operation(|| {
let mut firmware = sev::firmware::host::Firmware::open()
.map_err(|e| UserApiError::IOError(e))?;
let status = firmware.platform_status()?;
info!("플랫폼 상태: {:?}", status.state);
Ok(status)
}, "플랫폼 상태 조회");
match result {
Ok(_) => println!("작업 완료"),
Err(e) => {
comprehensive_error_handler(e);
return Err("SEV 작업 실패".into());
}
}
Ok(())
}
```

디버깅 도구 및 팁

1. 시스템 상태 확인 스크립트

```bash
#!/bin/bash

# sev_debug.sh - SEV 시스템 상태 확인


echo "=== SEV 디버그 정보 ==="


echo "1. 디바이스 파일 확인:"

ls -la /dev/sev /dev/kvm 2>/dev/null || echo "디바이스 파일 없음"


echo -e "\n2. 커널 모듈 상태:"

lsmod | grep -E "(kvm|sev)" || echo "SEV/KVM 모듈 로드되지 않음"


echo -e "\n3. CPU 기능 확인:"

grep -E "(svm|sev)" /proc/cpuinfo | head -5 || echo "SEV 기능 없음"


echo -e "\n4. 메모리 정보:"

free -h


echo -e "\n5. 커널 로그 (최근 10줄):"

dmesg | grep -i sev | tail -10 || echo "SEV 관련 로그 없음"


echo -e "\n6. 권한 확인:"

groups | grep -E "(kvm|root)" || echo "KVM 그룹 권한 없음"
```

2. Rust 디버그 매크로

```
macro_rules! debug_sev {
($operation:expr, $context:expr) => {
{
println!("🔍 디버그: {} 시작", $context);
let start = std::time::Instant::now();
let result = $operation;
let duration = start.elapsed();
match &result {
Ok(_) => println!("✅ 디버그: {} 성공 ({:?})", $context, duration),
Err(e) => {
println!("❌ 디버그: {} 실패 ({:?})", $context, duration);
println!(" 오류: {}", e);
}
}
result
}
};
}

// 사용 예제
fn main() -> Result<(), Box<dyn std::error::Error>> {
```

```
let mut firmware = debug_sev!(
sev::firmware::host::Firmware::open()
.map_err(|e| sev::error::UserApiError::IOError(e)),
"펌웨어 핸들 생성"
)?;
let _status = debug_sev!(
firmware.platform_status(),
"플랫폼 상태 조회"
)?;
Ok(())
}
```

이 오류 처리 가이드를 통해 AMD SEV 라이브러리 사용 시 발생할 수 있는 다양한 오류 상황을 효과적으로 처리하고 디버깅할 수 있습니다. 각 오류 타입에 대한 구체적인 해결 방법과 예방 조치를 참조하여 안정적인 SEV 애플리케이션을 개발하세요.